

DM Tools

Planning, Graph, and Dashboard Plan

A structured private planning workspace with strong organization and visual connections, without turning campaign play into administrative work.

DECISION	RECOMMENDATION
Core direction	Build one private planning workspace for threads, quests, scenarios, encounters, and free-form notes, all linked to existing campaign entities.
Structure	Folders organize navigation; typed links organize meaning; graphs and dashboards are derived views. Sessions never own planning records.
Restraint	Status and result tracking stay optional and shallow. Homebrew rules and combat calculation belong in later specialized addons.

Architecture review and phased implementation plan | 26 July 2026

1. Executive plan

Recommended destination

DM Tools should be a private planning knowledge base: structured enough to organize complex plots, encounters, NPC connections, and reusable possibilities, but lightweight enough that the DM never has to maintain a second history of the campaign.

What is already strong

- The host foundation is sufficient. DM-only collections, atomic multi-collection transactions, reviewed imports, lifecycle cleanup, graph facade, localization, and role-aware synchronization are already present.
- Core campaign entities are useful facts. Characters, factions, locations, events, mysteries, maps, and historical records should remain independent of planning tools.
- Addon boundaries are sound. DM Tools can work alone; sheets and compendiums can integrate through versioned APIs without becoming hard dependencies.

What currently blocks a coherent campaign flow

Gap	Why it matters	Design response
One flat scenario list	Notes, plots, quests, and encounters need different labels but share organization, linking, search, and graph behavior.	Use one discriminated planning-item model with a small set of kinds.
Organization and meaning can mix	A folder tree is useful for browsing but becomes brittle if it also defines story causality.	Keep folder hierarchy separate from typed semantic links.
Scenario graph has nodes but no edges	It is a catalog in graph form, not a planning aid.	Add explicit links, core-entity references, scoped filters, and backlinks.
Core facts could be duplicated	Copying NPC or location details into planning records creates stale parallel data.	Reference host entities and show live summaries instead of copying them.
No persisted schema migration path	The data model is still small, so this is the cheapest moment to establish safe evolution.	Version records and add additive, tested migrations before new UI.

Scope guardrails

- No database, framework, or infrastructure rewrite. JSON remains appropriate at this campaign scale.
- No session ownership, automatic progress, mandatory result logs, or expected scene order.
- No second copies of characters, factions, locations, mysteries, artifacts, maps, or historical events.
- No unlimited custom node types or edge semantics. New structure must justify its maintenance cost.
- Encounter preparation belongs here; encounter rules, difficulty math, and homebrew mechanics belong in specialized addons.
- DM planning stays private. Player-facing publication is explicit and deferred.

2. Planning workflow without session binding

Planning can be extensive without predicting the path of play. The workspace should help capture, organize, connect, and retrieve possibilities; it should not demand that the DM reconcile every plan against every session.



Planning is reusable and non-linear. No item is assigned to an expected session.

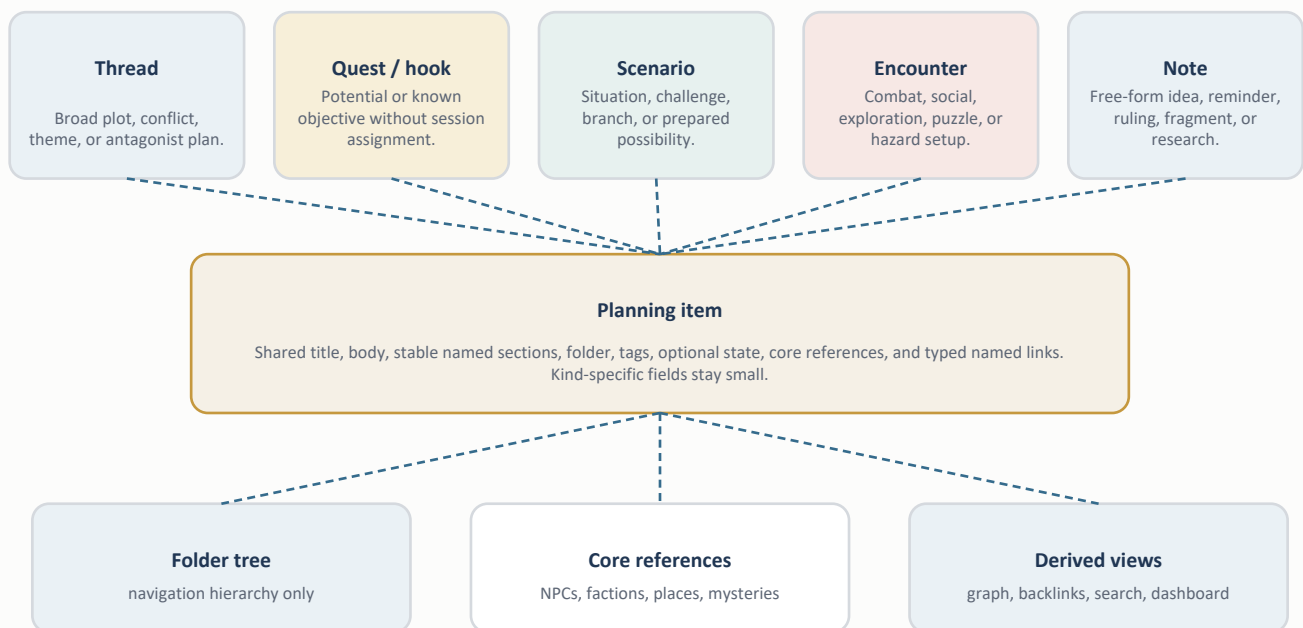
Stage	DM intent	Tooling that should support it	Durable result
Capture	Save an idea before it is lost.	Fast note creation with body-only input, optional title, current folder, and later classification.	An inbox item, not a form-filling interruption.
Organize	Build a navigable private reference.	Folders, tags, item kinds, pinning, search, archive, drag-and-drop placement.	A stable notebook structure.
Connect	See how plans touch the campaign.	Live references to NPCs, factions, places, mysteries, maps, other plan items, and encounters.	Backlinks and explicit graph edges.
Explore	Understand a complicated plan visually.	Scoped graphs, saved filters, edge legends, list fallback, and focus around one selected node.	A derived view, never duplicated truth.
Use and revise	Consult plans and adjust only what remains useful.	Quick open, one-click state changes, concise outcome note, optional core event backlink.	No mandatory session record or reconciliation.

Maintenance rule

The system may offer useful structure, but it must never nag the DM to complete it. A skipped quest, unused encounter, or radically changed situation remains valid reference material until manually revised, archived, or reused.

3. Proposed domain model

Use one common planning-item contract with a small kind discriminator. This keeps editing, search, import, graphing, and lifecycle code shared while allowing a few kind-specific fields where they genuinely help.

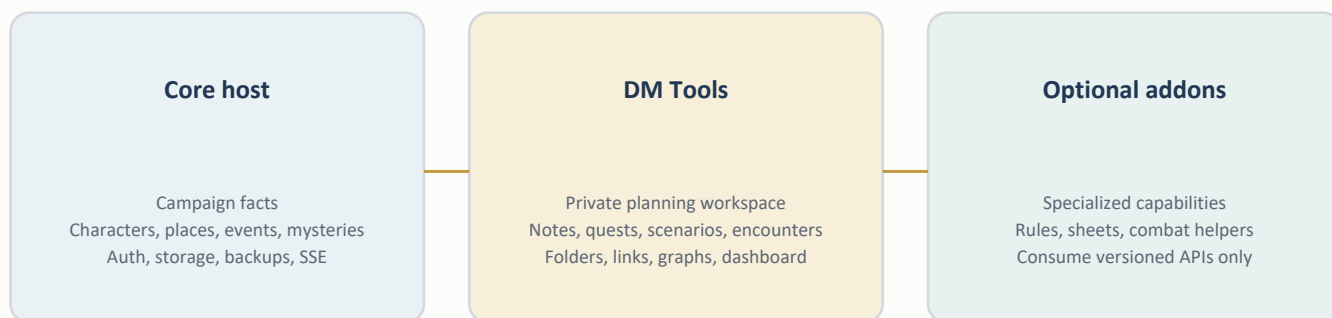


Concept	Owns	References	Lifecycle rule
Thread	Premise, stakes, broad plan, optional simple state	Quests, scenarios, encounters, NPCs, factions, places	An organizing lens, not a progress engine or mandatory root.
Quest / hook	Hook, objective, reward, possibilities, named parts, optional state	Threads, scenarios, encounters, mysteries, core entities	May be ignored, combined, changed, resolved unexpectedly, or abandoned.
Scenario	Situation, branches, discoveries, contingencies, reusable prose	Any planning items and core entities	Never assigned to a session and never requires ordered scenes.
Encounter	Kind, setup, participants, location, terrain, tactics, notes	Scenario, NPCs, locations, optional provider records	Reusable preparation; no combat or rules calculation inside DM Tools.
Note	Free-form body, tags, optional title and pin	Anything or nothing	Lowest-friction capture; promote to another kind only when useful.
Folder	Title, parent, display order	Planning items and child folders	Navigation only. Moving a folder never changes narrative meaning or graph edges.

Relationship discipline

- Use typed core references for involvement: character, faction, location, mystery, artifact, event, map, and optional addon record.
- Every item may contain optional stable named sections. A link endpoint can target the whole item or one section, so an NPC can connect directly to a specific quest part.
- Keep manual relation types small: related, supports, opposes, reveals, and requires. Every edge also carries a user-facing name such as Secret patron, Betrays here, or Knows the password.
- Requires means a genuine information or world-state precondition, not an expected session sequence.
- Folder placement, tags, timestamps, session numbers, and visual proximity never create semantic graph edges.

4. Tooling surfaces and ownership



Ownership is explicit. Integration is optional and contract-based.

Surface	Primary job	Show by default	Avoid
DM dashboard	Orient in under 30 seconds.	Pinned plans/views, current focus, ready encounters, linked entities, recent edits, secondary quick capture.	A session agenda, all records, or required maintenance.
Planning workspace	Build and reorganize extensive plans.	Folder tree, item list, tags, search, archive, drag-and-drop, keyboard navigation.	Using folders as story semantics or treating notes as the main product.
Planning editor	Edit all kinds consistently.	Title, body, named sections, kind, folder, tags, state, references, links, kind-specific fields.	A separate form architecture for every kind.
Connections / backlinks	Build and inspect precise relationships.	Named incoming/outgoing links targeting an item or section, live core summaries, broken-reference diagnostics.	Automatic inferred causality.
Planning graph	Understand complex structure visually.	Selected scope, item/section expansion, named edges, filters, legend, focus, list fallback.	Opening the entire campaign or every section by default.
Encounter editor	Prepare reusable challenges.	Participants, place, setup, terrain, tactics, outcomes, optional external records.	Rules adjudication, encounter balance, initiative, or combat state.
Import center	Bring in prepared structures safely.	Schema diagnostics, exact preview, atomic commit, stable IDs.	Implicit overwrite or a second editing workflow.
Optional history	Offer context when useful.	Core events and optional backlinks from used plans.	Mandatory results, progress journals, or DM-written session summaries.

Navigation recommendation

- Keep the stable /dm dashboard route as the entry point.
- Use one /dm-plans workspace with folder, list, editor, and graph modes; preserve the existing scenario route as a filtered deep link or redirect.
- Let every planning item, core entity, backlink list, and saved view deep-link to a focused graph.
- Treat the graph as a view mode within a domain page, not a separate source of truth.
- Keep core entity navigation in the host; DM Tools links into it using existing routes and collection descriptors.

5. Storage, contracts, and safe evolution

The main architectural risk is multiplying collections and editors every time a new planning concept appears. A common planning-item model keeps the addon small while discriminated validation prevents it from becoming an untyped document dump.

Collection	Shape	Access	Notes
planningItems	Keyed	DM-only	Common fields plus stable named sections and kind: thread, quest, scenario, encounter, or note.
planningFolders	Keyed	DM-only	Parent, title, display order, and cycle checks; navigation only.
planningLinks	Keyed	DM-only	Named typed edges; endpoints may target an item, its stable section, or a permitted core entity.
planningViews	Keyed, later	DM-only	Named filter/focus definitions only. Persist graph positions only if local layouts prove insufficient.
scenarios	Legacy list	DM-only	Migrate once into planningItems with kind scenario; preserve IDs and import compatibility.

Record evolution

- Add schemaVersion to persisted planning records before introducing new shapes.
- Ship additive, deterministic migrations with fixtures for old, mixed, partial, and repeated-startup states.
- Keep import schema versioning separate from persisted schema versioning. Imports describe input; migrations protect stored campaign data.
- Use host transactions for migrations and edits that create or remove items, folders, and links together.
- Centralize kind definitions, section-anchor rules, optional states, relation types, edge names, reference validation, and display metadata so every consumer agrees.

Visibility and publication

Private planning is the default boundary

DM Tools should never expose planning items merely because they reference a public character or location. If a future feature publishes a quest card or handout, it creates an explicit public projection containing approved fields only; the source plan and its links remain private.

Integration contract

- DM Tools remains independently useful and has no hard dependency on sheets or the compendium.
- External encounter references use provider ID, record kind, record ID, and a fallback label so plans remain readable when the provider is absent.
- Expose a small versioned read API only after the model stabilizes: planning item summaries, references, and encounter preparation fields.
- Optional consumers may enrich views but must degrade cleanly when absent. Homebrew rules, encounter balance, combat, and sheet calculations stay outside DM Tools.

6. Graph, encounter, and maintenance boundaries

At expected campaign sizes, JSON and browser rendering are sufficient. Readability and maintenance cost will fail before storage scale, so every complex feature needs a narrow scope and an escape hatch.

Concern	Recommendation	Reason
Graph growth	Open on a selected folder, thread, item, entity, or saved filter; collapse sections by default, expand them on demand, and keep a list.	Section nodes provide precision but would quickly overwhelm an unscoped graph.
Dashboard growth	Show pinned items/views, recent edits, quick capture, ready encounters, and small core summaries. Link to full pages.	The dashboard should orient, not become another planner screen.
Change detection	Use collection revisions or a deterministic content digest, not counts plus newest timestamp.	Equivalent counts/timestamps can hide meaningful edits.
Reference integrity	Diagnose dangling IDs, duplicate IDs, folder cycles, invalid kinds/states, broken section anchors, links, and providers without blocking unrelated content.	Planning graphs amplify small reference errors into confusing visual failures.
External references	Keep a fallback label and visibly mark unavailable provider records; never delete the plan when an addon disappears.	Compendium, sheets, and future rules addons remain optional.
Encounter boundary	Store setup and intent, not calculated difficulty or runtime combat state.	Planning is edition-neutral; mechanics belong to a rules-aware addon.
Graph fallback	Keep the accessible list usable when Cytoscape is missing, unsupported, or fails.	Graph availability must not remove core planning functionality.

Operational invariants

- Every ID is stable across imports, migration, reinstall, backup, and restore.
- Moving an item or folder changes navigation only; typed links and core references remain unchanged.
- Every cross-record reference is validated on write and reported on read if legacy or external data is inconsistent.
- Every multi-record item/folder/link update is atomic: either all selected updates publish or none do.
- Every derived graph can be recreated from persisted records; graph layout is presentation state only.
- Every optional integration has a usable absent-provider state.
- Every optional state transition has an explicit user action; no missing update creates a warning or blocks use.

Deferred until demanded by real play

- Custom item kinds, unlimited relation types, automatic quest progress, session plans, and predictive analytics.
- Encounter balance, initiative, runtime combat state, or homebrew-rule enforcement inside DM Tools.
- Player publication, session summaries, and result journals until actual use demonstrates value.
- Server-side graph positions and collaborative cursors until local layouts and named filters prove insufficient.

7. Phased implementation plan

The order is intentionally dependency-driven: define and protect the model before building screens that would freeze accidental semantics.

1

Domain foundation and migration

Build: Canonical kind, stable-section, reference, and named-link contracts; schemaVersion; three collections; migrate scenarios with stable IDs; diagnostics.

Gate: Old data survives restart and backup/restore; invalid references are reported; no UI behavior regresses.

2

Planning workspace

Build: Quick capture, folder tree, shared editor, stable named sections, kinds, tags, pin/archive, search, drag-and-drop, draft protection, import update.

Gate: The DM can organize a substantial real plan without raw JSON, duplicate entities, or mandatory fields.

3

Connections and graph

Build: Core references, item/section link endpoints, named edges, backlinks, collapsed/expanded graph nodes, focus, filters, legend, list fallback.

Gate: A complex plot can be understood visually without opening a whole-campaign hairball.

4

Encounter planning

Build: Encounter-specific fields, NPC/location references, optional provider records with fallback labels, reusable templates, graph participation.

Gate: Combat, social, exploration, puzzle, and hazard setups work without importing rules logic into DM Tools.

5

Dashboard and saved focus

Build: Quick capture, pinned items/views, recent edits, ready encounters, unresolved-mystery summary, shortcuts, optional named graph filters.

Gate: The dashboard is useful every time it opens but never demands campaign bookkeeping.

6

Optional history and integrations

Build: Event backlinks, derived player session history if wanted, versioned planning API, optional provider enrichment, future explicit publication.

Gate: Removing any optional addon leaves plans readable; no retrospective workflow becomes mandatory.

8. Decisions, tradeoffs, and acceptance criteria

Decision	Recommendation	Tradeoff accepted
Planning model	One discriminated planning-item collection with five kinds and optional stable named sections.	Kinds share infrastructure; strict validation is required to avoid a loose catch-all schema.
Organization	Folders define navigation; typed links and references define meaning; graphs derive from links.	Users maintain two distinct concepts, but moving notes never corrupts story relationships.
Session relationship	No plan belongs to a session and no retrospective update is required.	There is no dedicated DM run sheet or automatic campaign-progress report.
State tracking	Optional shallow states only, with no percentages, clocks, or automatic transitions.	Less automation, but almost no bookkeeping and no false precision.
Encounter boundary	Store reusable setup and intent; delegate rule calculations and live combat to specialized addons.	Some encounter enrichment is unavailable without a rules provider.
Integration	Soft, versioned, capability-based APIs and fallback labels for external records.	Providers can disappear without data loss, at the cost of visibly degraded enrichment.

Release acceptance criteria

- Understandable: a new maintainer can explain planning kinds, folders, typed links, core references, and derived views without reading implementation code.
- Recoverable: migration, transaction failure, addon disable/re-enable, restart, backup, and restore preserve campaign data and stable IDs.
- Fast to capture: a body-only note can be saved in one action and organized later.
- Organizable: hundreds of notes can be navigated through folders, tags, search, pinning, backlinks, and archive without a new schema.
- Agency-safe: skipping, combining, unexpectedly resolving, or abandoning plans never invalidates data or forces record reorganization.
- Graphable: an NPC can have a named link to a whole quest or one named quest part; collapsed and expanded graphs preserve that meaning.
- Optional: DM Tools works without sheets, compendium, or future combat tooling; integrations fail closed and degrade visibly.

Recommended immediate next batch

Implement Phase 1 only. Establish planning items, stable named sections, folders, named item/section links, references, migrations, and integrity contracts before UI work. Preserve scenario data; add no session ownership, automatic progress, or homebrew-rule fields.

Review basis

This plan is based on a source-level audit of the host and DM Tools contracts, current planning/import/graph/dashboard implementations, reference documentation, and the local campaign data shape. It does not sample either remote production campaign. No campaign names, prose, record IDs, or personal data are included.